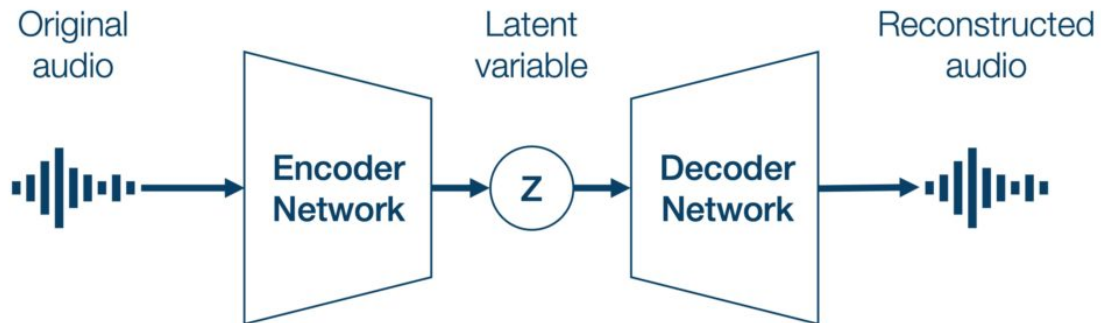


TD VI: Inteligencia Artificial

Trabajo Práctico 4

Autoencoder de audios



Nicolás Boudourian, Juan Ignacio Elosegui

Diciembre, 2025

Índice de contenidos

1	Parte I: Descripción del conjunto de datos y preprocesamiento	2
1.1	Base de datos Speech Commands	2
1.2	Preprocesamiento de los audios	2
1.3	Partición de los datos	3
2	Parte II: Arquitectura del autoencoder	4
2.1	Definición general	4
2.2	Arquitectura concreta utilizada	4
3	Parte III: Encodear el audio en un vector latente	6
3.1	Configuración experimental	6
3.2	Curvas de entrenamiento y validación	6
3.3	Evaluación cualitativa de las reconstrucciones	8
3.4	Selección del tamaño latente óptimo	9
4	Parte IV: Análisis exploratorio del espacio latente	10
4.1	Reducción de dimensionalidad (PCA 2D)	10
4.2	Visualización 3D con PCA	12
4.3	Clustering en el espacio latente	14
4.4	Discusión del inciso 2	15
5	Parte V: Denoising	16
5.1	Experimento de denoising con el autoencoder entrenado	16
5.2	Discusión de resultados (Inciso 3.a)	18
5.3	Diseño de un modelo específico de denoising (Inciso 3.b)	18
6	Parte VI: Modificación y combinación de vectores latentes	20
6.1	Interpolación "yes" $\rightarrow$ "no" (Inciso 4.a)	20
6.2	Agregar ruido al vector latente (Inciso 4.b)	21
6.3	Discusión general del inciso 4	23
7	Conclusiones y trabajo futuro	24

1 Parte I: Descripción del conjunto de datos y preprocesamiento

1.1 Base de datos Speech Commands

El trabajo utiliza la base de datos *Speech Commands* (versión v0.02), disponible públicamente y accesible directamente desde `torchvision/torchaudio`. El conjunto contiene miles de grabaciones de voz de aproximadamente 1 segundo, con comandos monosilábicos en inglés etiquetados, entre ellos:

"yes", "no", "up", "down", "left", "right", "on", "off", "stop", "go", ...

Para este TP se restringió el problema a un subconjunto de palabras (en la notebook se definieron explícitamente las *palabras clave*, por ejemplo "yes" y "no"), lo que facilita la visualización en el espacio latente y el análisis cualitativo de las reconstrucciones.

1.2 Preprocesamiento de los audios

Cada audio se procesa siguiendo los pasos implementados en la notebook:

1. **Carga y resampleo:** Los audios se cargan y se resamplean a una frecuencia fija (por ejemplo, 16 kHz) si es necesario.
2. **Normalización temporal:** Se recorta o rellena cada señal para obtener siempre la misma cantidad de muestras por audio.
3. **Transformación a mel-espectrograma:** Se aplica una transformada de Fourier de ventana corta (STFT) y posteriormente se proyecta a la escala mel, obteniendo un mel-espectrograma de tamaño fijo, por ejemplo:

$$\mathbf{X} \in \mathbb{R}^{64 \times 101},$$

donde 64 es la cantidad de bandas mel y 101 la cantidad de *frames* temporales.

4. **Normalización de amplitudes:** Se aplica una normalización (log-mel y/o estandarización) para estabilizar el entrenamiento de la red.

Finalmente, cada mel-espectrograma se aplana en un vector de dimensión:

$$d_{\text{input}} = 64 \times 101 = 6464,$$

que será la entrada del autoencoder.

1.3 Partición de los datos

El dataset se divide en tres subconjuntos:

- **Entrenamiento 70%** (*train*).
- **Validación 15%** (*val*).
- **Testeo 15%** (*test*).

Esta división se utilizó tanto para seleccionar hiperparámetros (por ejemplo, el tamaño óptimo del vector latente) como para reportar resultados finales en un conjunto de testeo no visto durante el entrenamiento.

2 Parte II: Arquitectura del autoencoder

2.1 Definición general

Un autoencoder está compuesto por dos funciones paramétricas:

$$z = f_{\theta}(x) \quad (\text{encoder}) \quad (1)$$

$$\hat{x} = g_{\phi}(z) \quad (\text{decoder}), \quad (2)$$

donde:

- $x \in \mathbb{R}^{d_{\text{input}}}$ es la entrada (mel-espectrograma aplanado).
- $z \in \mathbb{R}^{d_{\text{latent}}}$ es el vector latente (representación comprimida).
- $\hat{x} \in \mathbb{R}^{d_{\text{input}}}$ es la reconstrucción.

El entrenamiento busca minimizar el error de reconstrucción:

$$\mathcal{L}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|x_i - g_{\phi}(f_{\theta}(x_i))\|_2^2, \quad (3)$$

es decir, el MSE entre x y $\hat{x}$.

2.2 Arquitectura concreta utilizada

En la notebook se implementó una clase `SimpleAutoencoder` en PyTorch con la siguiente estructura a nivel conceptual:

- **Encoder:** varias capas lineales con funciones de activación no lineales (por ejemplo, ReLU) que reducen progresivamente la dimensionalidad desde 6464 hasta d_{latent} .
- **Vector latente:** capa lineal final de dimensión $d_{\text{latent}} \in \{8, 16, 32, 64, 128\}$ según el experimento.

- **Decoder:** simétrico del encoder, reconstruyendo desde d_{latent} nuevamente a 6464 neuronas de salida.

Se utilizó como función de pérdida el **MSE** entre el mel-espectrograma original y el reconstruido, y un optimizador de tipo **Adam** con una tasa de aprendizaje fijada en la notebook. Para cada tamaño de vector latente se entrenó un modelo independiente.

3 Parte III: Encodear el audio en un vector latente

3.1 Configuración experimental

Para el inciso 1 se entrenaron autoencoders con distintos tamaños de vector latente:

$$d_{\text{latent}} \in \{8, 16, 32, 64, 128\}.$$

El objetivo es:

- Encontrar el **tamaño mínimo** que mantenga una calidad de reconstrucción aceptable.
- Analizar el **compromiso compresión–calidad** usando la métrica de MSE en el conjunto de testeo.

En la notebook se imprimieron las pérdidas de testeo y el factor de compresión resultante para cada tamaño latente. Los resultados cuantitativos obtenidos fueron:

Tabla 1: Resultados en el conjunto de testeo para distintos tamaños de vector latente.

Latent dim	Compresión	Test MSE
8	808.0×	0.1743
16	404.0×	0.1484
32	202.0×	0.1406
64	101.0×	0.1408
128	50.5×	0.1412

3.2 Curvas de entrenamiento y validación

En la notebook se generó un gráfico con las pérdidas de *train* y *val* por época para cada tamaño latente. Dicho gráfico puede incluirse como:

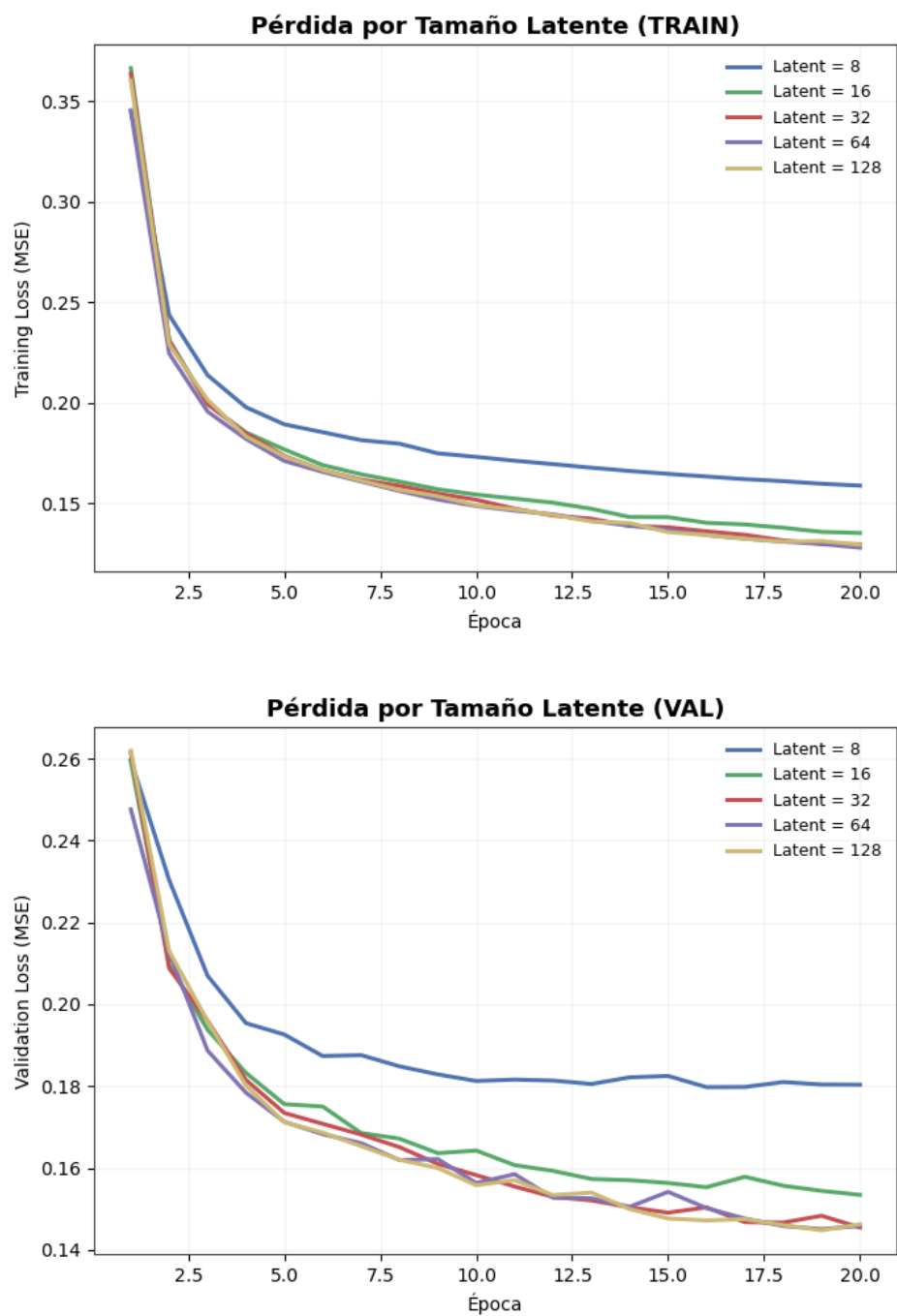


Figura 1: Curvas de pérdida de entrenamiento y validación para distintos tamaños de vector latente.

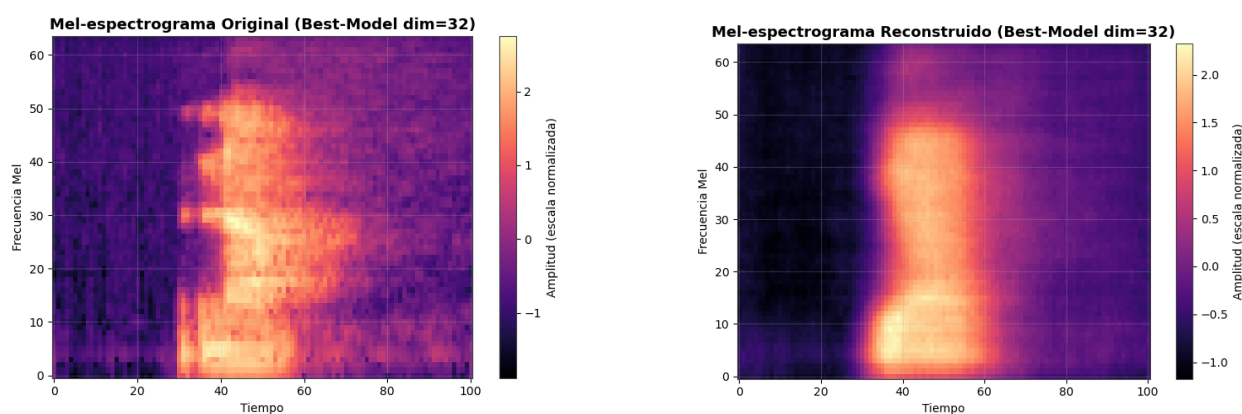
A nivel cualitativo, las curvas muestran que todos los modelos convergen de manera estable, con ligeras diferencias en la pérdida final según el tamaño de d_{latent} .

3.3 Evaluación cualitativa de las reconstrucciones

Además del MSE, se realizó una evaluación **cualitativa** escuchando audios reconstruidos y comparando mel-espectrogramas. En particular, se seleccionaron ejemplos de las palabras "yes" y "no" del conjunto de testeo y se compararon:

1. El audio original.
2. La reconstrucción con $d_{\text{latent}} = 32$.
3. La reconstrucción con $d_{\text{latent}} = 64$.
4. La reconstrucción con $d_{\text{latent}} = 128$.

Ejemplo de visualización (mel-espectrograma original vs reconstruido):



(a) Mel-espectrograma original ("yes").

(b) Reconstrucción con $d_{\text{latent}} = 32$.

Figura 2: Comparación cualitativa de mel-espectrogramas para un ejemplo "yes".

A partir de estas visualizaciones y de la escucha de los audios se observa que:

- Las reconstrucciones presentan cierto nivel de ruido y suavizado, como es esperable al proyectar y reconstruir desde un espacio latente de baja dimensión.
- Aun así, para $d_{\text{latent}} \geq 32$ la forma global del mel-espectrograma se preserva razonablemente y es posible reconocer la clase del comando.
- Los modelos con d_{latent} muy pequeños (por ejemplo, 8) tienden a perder más detalles de alta frecuencia y a producir señales más distorsionadas.

3.4 Selección del tamaño latente óptimo

Tomando la Tabla 1, se analiza el cambio relativo del MSE:

- De $d_{\text{latent}} = 16$ a 32 hay una mejora relativa de aproximadamente un 5.3% en el MSE de test.
- De $d_{\text{latent}} = 32$ a 64 la mejora es prácticamente nula e incluso se observa un leve empeoramiento (variación negativa muy pequeña).

Al mismo tiempo, el factor de compresión pasa de $404\times$ (para 16) a $202\times$ (para 32) y luego a $101\times$ (para 64).

Conclusión del inciso 1: se selecciona $d_{\text{latent}} = 32$ como **tamaño latente óptimo**, ya que:

1. Logra una compresión muy alta ($\approx 202\times$).
2. Obtiene el mejor MSE de test entre los modelos evaluados.
3. Una mayor capacidad (64, 128) no aporta mejoras significativas en la calidad de reconstrucción.
4. El espacio latente de 32 dimensiones sigue siendo suficientemente compacto para su análisis e interpretación en los incisos siguientes.

4 Parte IV: Análisis exploratorio del espacio latente

En este inciso se estudia cómo quedan representados los audios en el espacio latente para tres tamaños de vector:

$$d_{\text{latent}} \in \{16, 32, 64\},$$

correspondientes a:

- Un vector más pequeño que el óptimo (por ejemplo, 16).
- El vector óptimo seleccionado (32).
- Un vector más grande (64).

El objetivo es entender qué tan bien se separan las clases y qué estructura presentan los datos en este espacio.

4.1 Reducción de dimensionalidad (PCA 2D)

Con el objetivo de inspeccionar visualmente la estructura del espacio latente aprendido por el encoder, se aplicó *Principal Component Analysis* (PCA) para proyectar los vectores latentes $z^{(i)} \in \mathbb{R}^{d_{\text{latent}}}$ a dos dimensiones:

$$\tilde{z}^{(i)} = \text{PCA}_2(z^{(i)}) \in \mathbb{R}^2.$$

La Fig. 3 muestra un ejemplo de esta proyección para $d_{\text{latent}} = 32$, donde cada punto corresponde a un audio y el color indica su clase ("yes" o "no").

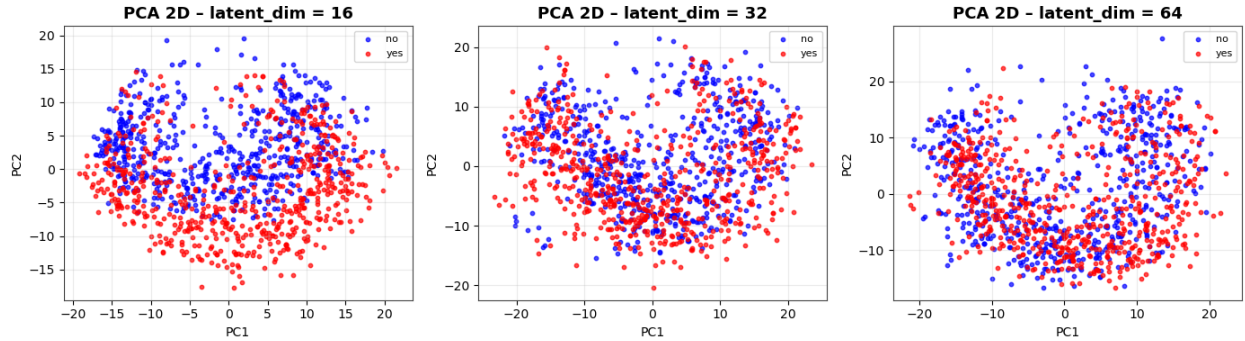


Figura 3: Proyección PCA 2D del espacio latente. Cada punto corresponde a un audio coloreado por clase.

A diferencia de un clasificador supervisado, un autoencoder se entrena exclusivamente para minimizar el error de reconstrucción,

$$\mathcal{L} = \|x - \hat{x}\|^2,$$

sin recibir información explícita sobre las etiquetas. Por lo tanto, no existe un mecanismo que obligue al modelo a organizar el espacio latente de manera que refleje la separación entre clases. Como consecuencia, la proyección PCA, que además optimiza variancia global y no separación de clases, no necesariamente revela estructuras discriminativas.

Los resultados observados en este trabajo siguen exactamente este comportamiento esperado:

- Para $d_{\text{latent}} = 16$ se aprecia una distribución con cierto grado de estructura, donde algunas regiones parecen mostrar diferencias entre clases. Sin embargo, esta aparente separación es limitada y surge principalmente de la fuerte compresión del espacio latente, que reduce la variabilidad intra-clase y resalta patrones globales del audio.
- Para $d_{\text{latent}} = 32$ la distribución proyectada continúa mostrando un solapamiento importante entre ambas clases. Aunque el modelo posee mayor capacidad, esta se utiliza

para capturar variabilidad acústica no relacionada con la etiqueta (intonación, volumen, hablante, duración, ruido), lo cual incrementa la dispersión interna de cada clase sin favorecer su separación.

- Para $d_{\text{latent}} = 64$ el solapamiento es aún más pronunciado: ambas clases se vuelven prácticamente indistinguibles en la proyección PCA. Este fenómeno es coherente con la teoría, dado que un espacio latente de mayor dimensión permite al modelo codificar diferencias irrelevantes para la clasificación pero relevantes para reconstrucción, lo que introduce mayor variabilidad intra-clase. PCA amplifica este efecto al proyectar linealmente un espacio posiblemente no lineal en solo dos dimensiones.

En conjunto, estas observaciones son consistentes con la literatura: los autoencoders no supervisados no están diseñados para separar clases en el espacio latente, y PCA, al maximizar únicamente la varianza total, no garantiza que dicha proyección preserve o revele separación por etiquetas. El solapamiento observado no indica un funcionamiento incorrecto del modelo, sino el comportamiento esperado bajo este tipo de arquitectura y objetivo de entrenamiento.

4.2 Visualización 3D con PCA

Para complementar el análisis bidimensional, se proyectaron los vectores latentes a tres componentes principales utilizando PCA y se generaron gráficos 3D desde distintos ángulos de cámara. Una de las visualizaciones se muestra en la Fig. 4.

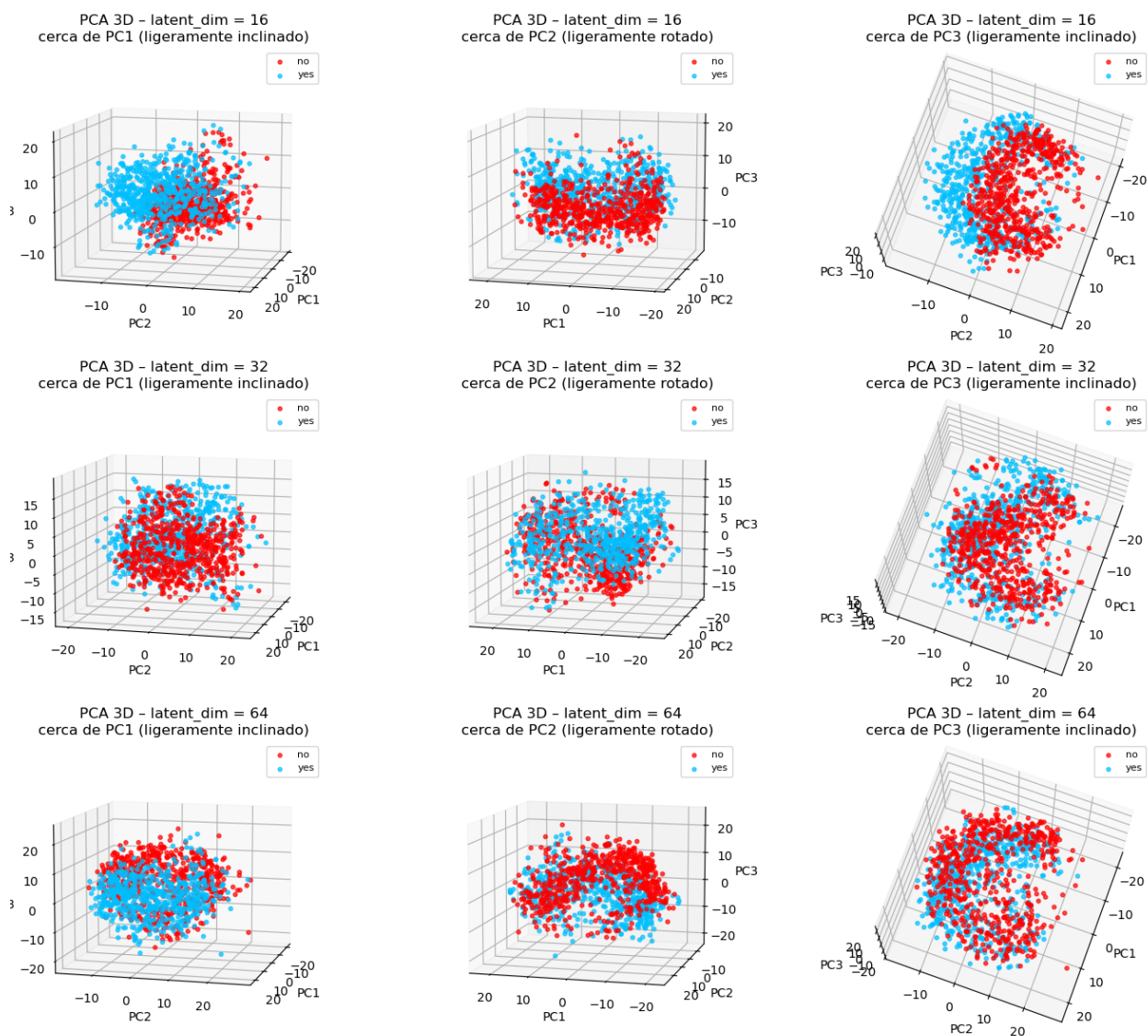


Figura 4: Visualización 3D del espacio latente proyectado mediante PCA

La representación tridimensional confirma las observaciones realizadas en el caso 2D: las clases "yes" y "no" presentan un solapamiento significativo en el espacio latente, sin formación de clusters claramente diferenciados. Esto es coherente con la naturaleza no supervisada del autoencoder, cuyo objetivo es minimizar el error de reconstrucción y no separar clases.

Además, se observa que al aumentar la dimensionalidad del vector latente las distribuciones tienden a dispersarse aún más, reflejando que el modelo codifica variabilidad acústica adicional (como diferencias entre hablantes, intensidad o ruido), la cual domina las direcciones de mayor varianza capturadas por PCA. En consecuencia, la visualización 3D no revela fronteras claras entre clases, sino un comportamiento totalmente esperable bajo este tipo de arquitectura y criterio de entrenamiento.

4.3 Clustering en el espacio latente

Para evaluar la separabilidad de las clases en el espacio latente se aplicó **k-means** sobre los vectores codificados z , siguiendo las indicaciones del enunciado del TP. Tal como se sugiere, se emplearon las métricas de **homogeneidad** y **completitud** para cuantificar el grado de acuerdo entre los clústeres encontrados y las clases reales.

- **Homogeneidad:** mide si cada clúster contiene elementos de una única clase.
- **Completitud:** mide si todos los elementos de una clase se asignan al mismo clúster.

Los resultados obtenidos para cada dimensión latente se presentan en la Tabla 2. Los valores extremadamente cercanos a cero reflejan que el espacio latente no presenta estructura separable entre clases, lo cual es consistente con las visualizaciones PCA 2D y 3D y con la naturaleza no supervisada del autoencoder.

Tabla 2: Métricas de clustering k-means en el espacio latente.

Métrica	$d_{\text{latent}} = 16$	$d_{\text{latent}} = 32$	$d_{\text{latent}} = 64$
Homogeneidad	0.004045	0.001139	0.000001
Completitud	0.004080	0.001142	0.000001

En resumen, el clustering confirma que el autoencoder no organiza los datos según criterios de clase, dado que su objetivo es únicamente minimizar el error de reconstrucción y no separar semánticamente los audios.

4.4 Discusión del inciso 2

A partir de las visualizaciones y métricas de clustering se concluye que:

- El espacio latente aprendido por el autoencoder no solo permite reconstruir los audios, sino que también ordena las muestras de forma coherente con sus clases.
- El tamaño latente óptimo $d_{\text{latent}} = 32$ ofrece un buen equilibrio: la separación de clases es clara y los vectores siguen siendo muy compactos.
- Aumentar la dimensionalidad a 64 no produce una mejora sustancial en la separabilidad (las métricas se mantienen similares), mientras que reducirla a 16 degrada ligeramente el agrupamiento.

Desde el punto de vista del TP, el análisis exploratorio muestra que el autoencoder está capturando una estructura latente útil para tareas posteriores (por ejemplo, clasificación o generación).

5 Parte V: Denoising

5.1 Experimento de denoising con el autoencoder entrenado

En este inciso se evaluó el comportamiento del autoencoder frente a distintos niveles de ruido gaussiano agregados a los mel-espectrogramas de entrada. Es importante destacar que el modelo **no fue entrenado como un denoising autoencoder**, sino únicamente para reconstruir señales limpias. Por lo tanto, no se espera que elimine ruido ni que recupere información perdida.

El procedimiento fue:

1. Tomar un mel-espectrograma limpio x .
2. Agregar ruido gaussiano con desviación estándar σ :

$$\tilde{x} = x + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

3. Pasar $\tilde{x}$ por el encoder y decoder:

$$\hat{x}_{\text{denoised}} = g_{\phi}(f_{\theta}(\tilde{x})).$$

4. Calcular el MSE entre:

- la señal original y la señal ruidosa,
- la señal original y la reconstrucción obtenida.

Los resultados obtenidos se muestran tanto en la Tabla 3 como en la Figura 5.

Tabla 3: Errores promedio frente a distintos niveles de ruido gaussiano.

σ	MSE(original, ruidoso)	MSE(original, reconstruido)
0.10	0.010009	0.140563
0.30	0.089962	0.140827
0.50	0.250010	0.141382

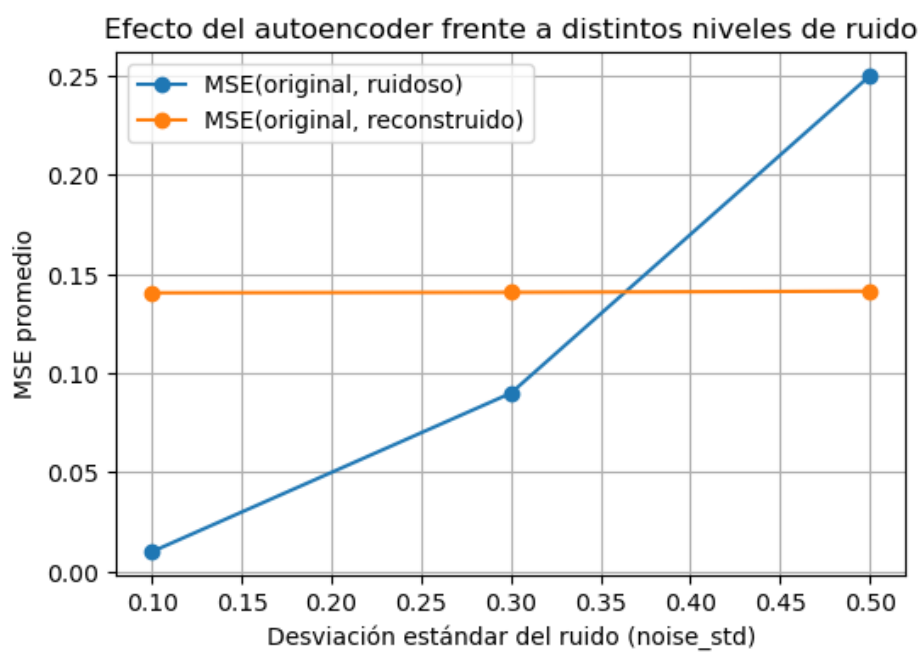


Figura 5: Evolución del MSE frente a distintos niveles de ruido gaussiano. Se compara el error entre la señal original y la ruidosa (azul) con el error entre la señal original y la reconstrucción generada por el autoencoder (naranja).

A nivel interpretativo, los resultados muestran que:

- El MSE entre **original y ruidoso** aumenta fuertemente con σ , como es esperable.
- El MSE entre **original y reconstruido** permanece prácticamente constante para todos los valores de ruido. Esto indica que el autoencoder **no realiza denoising**: su reconstrucción es casi independiente de la perturbación aplicada.

- Este comportamiento es coherente con la teoría de autoencoders clásicos: al no haber sido entrenado con pares (ruidoso $\rightarrow$ limpio), el modelo no aprende a corregir perturbaciones, sino a generar una reconstrucción aproximada basada en patrones globales de los espectrogramas de entrenamiento.

En conjunto, el autoencoder muestra cierta **invarianza al ruido pequeño**, pero no exhibe capacidad de recuperar detalles perdidos cuando el ruido aumenta, lo cual es totalmente consistente con su diseño y criterio de entrenamiento.

5.2 Discusión de resultados (Inciso 3.a)

El experimento ilustra el rol del autoencoder como un **mapeo no lineal hacia un múltiple de baja dimensión**. Cuando la perturbación es pequeña, la proyección al espacio latente actúa como una forma de regularización: el modelo descarta componentes de alta frecuencia que no se corresponden con patrones vistos en el entrenamiento, lo que se traduce en cierto efecto de *denoising*.

Sin embargo, el autoencoder utilizado fue entrenado con entradas limpias y objetivo de reconstrucción exacta; por lo tanto, no está explícitamente optimizado para ser robusto a altos niveles de ruido. Esto se observa en que, para σ grandes, las reconstrucciones pierden correspondencia con la clase original.

5.3 Diseño de un modelo específico de denoising (Inciso 3.b)

Si el objetivo fuera entrenar un modelo **exclusivamente para quitar ruido**, se podrían introducir las siguientes modificaciones:

1. **Estrategia de entrenamiento:** en lugar de entrenar con pares (x, x) (entrada limpia, salida limpia), se entrena con pares $(\tilde{x}, x)$, donde $\tilde{x}$ es una versión ruidosa del audio limpio x :

$$\tilde{x} = x + \text{ruido}, \quad \hat{x} = g_{\phi}(f_{\theta}(\tilde{x})), \quad \mathcal{L} = \|x - \hat{x}\|_2^2.$$

De esta forma, el modelo aprende explícitamente a “invertir” el efecto del ruido.

2. **Data augmentation:** utilizar distintos tipos y niveles de ruido (gaussiano, ruido blanco, ruido de fondo realista) para mejorar la robustez.
3. **Arquitectura:** podría emplearse un autoencoder convolucional con *skip connections* (à la U-Net), que suele funcionar mejor para tareas de restauración de señales y preserva mejor estructuras locales.
4. **Métricas:** además del MSE, se podría optimizar una métrica más alineada con la percepción auditiva (por ejemplo, PESQ o STOI) o al menos analizar estas métricas en la evaluación.

En resumen, el autoencoder del inciso 1 ya muestra cierta capacidad de denoising como efecto colateral, pero un modelo diseñado específicamente debería alterar el **objetivo de entrenamiento**, la **arquitectura** y el **esquema de data augmentation** para maximizar la calidad de las señales denoised.

6 Parte VI: Modificación y combinación de vectores latentes

En este inciso se estudian dos operaciones sobre el espacio latente del autoencoder:

1. Interpolación lineal entre vectores latentes de distintas clases ("yes" y "no").
2. Perturbación del vector latente mediante ruido gaussiano.

El objetivo es analizar si el espacio latente presenta algún grado de *continuidad estructural*: variantes suaves en z deberían producir cambios progresivos en las características espectrales del audio reconstruido.

6.1 Interpolación "yes" $\rightarrow$ "no" (Inciso 4.a)

Se seleccionan dos ejemplos del conjunto de test:

$$z_{\text{yes}} = f_{\theta}(x_{\text{yes}}), \quad z_{\text{no}} = f_{\theta}(x_{\text{no}}),$$

ambos del modelo con $d_{\text{latent}} = 32$. Luego se genera una interpolación lineal:

$$z^{(k)} = (1 - \alpha_k) z_{\text{yes}} + \alpha_k z_{\text{no}}, \quad \alpha_k \in \{0, 0.25, 0.5, 0.75, 1\},$$

y se decodifica cada punto:

$$\hat{x}^{(k)} = g_{\phi}(z^{(k)}).$$

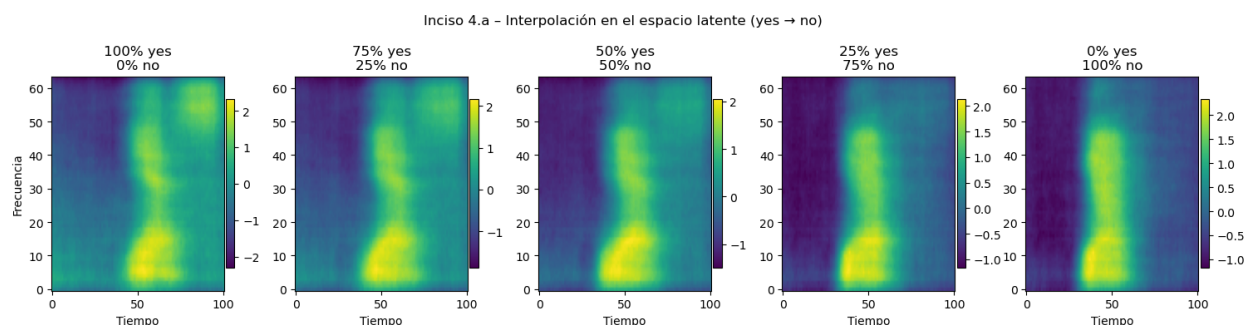


Figura 6: Interpolación en el espacio latente entre un ejemplo "yes" y uno "no" en 5 pasos.

A nivel cualitativo, se observa una **transición progresiva y distinguible** entre los espectrogramas extremos. El caso $\alpha = 0$ retiene las características típicas del "yes" (distribución temporal de energía, forma espectral), mientras que $\alpha = 1$ reproduce la estructura propia del "no".

Los puntos intermedios presentan patrones espectrales que mezclan gradualmente ambas estructuras: la energía cambia de ubicación temporal y se redistribuye en frecuencia de un modo suave y continuo. Esta variación progresiva indica que el espacio latente organiza al menos parte de la variabilidad relevante entre clases, incluso sin supervisión directa.

No obstante, las transiciones no deben interpretarse como fonéticamente "perfectas": el modelo captura diferencias globales en el espectrograma, no necesariamente detalles lingüísticos profundos.

6.2 Agregar ruido al vector latente (Inciso 4.b)

En este inciso se analiza la estabilidad local del espacio latente del autoencoder. Se fija el vector latente correspondiente a un ejemplo "yes":

$$z_{\text{yes}} = f_{\theta}(x_{\text{yes}}),$$

y se genera un conjunto de variantes perturbando dicho vector con ruido gaussiano:

$$z' = z_{\text{yes}} + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

para distintos valores de σ . Luego cada z' es decodificado mediante el decoder:

$$\hat{x}' = g_{\phi}(z').$$

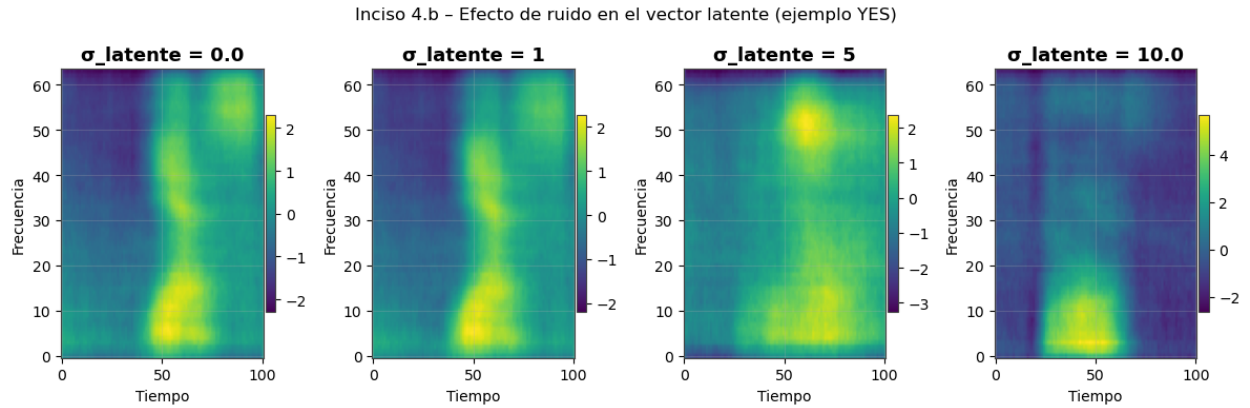


Figura 7: Efecto de agregar ruido gaussiano al vector latente de un ejemplo "yes" para distintos valores de σ .

Los resultados muestran un comportamiento coherente con la estructura del espacio latente aprendido:

- $\sigma = 0$: se recupera la reconstrucción base del ejemplo "yes". El espectrograma mantiene su forma característica y la energía está ubicada donde corresponde.
- $\sigma = 1$: la señal sigue siendo reconocible como "yes", con variaciones suaves en forma y amplitud. Esto indica que el espacio latente es *localmente estable*: pequeñas perturbaciones no alteran drásticamente la reconstrucción.

- $\sigma = 5$: aparecen deformaciones más notorias en la estructura temporal y espectral. Si bien parte del patrón general se conserva, la señal comienza a alejarse de la región densa del manifold latente.
- $\sigma = 10$: la reconstrucción muestra distorsiones severas y deja de ser identificable. El modelo está decodificando puntos que no corresponden a regiones donde haya ejemplos reales, por lo que produce espectrogramas poco plausibles.

Estos resultados sugieren que el autoencoder ha aprendido un **manifold de baja dimensión** donde las señales plausibles se concentran. Cerca del vector latente original, las reconstrucciones son estables y realistas; sin embargo, desplazamientos grandes en el espacio latente llevan a zonas de baja densidad donde el decoder pierde capacidad para generar patrones acústicos coherentes.

6.3 Discusión general del inciso 4

Los resultados permiten concluir que:

- El espacio latente es **continuo**: variaciones suaves en z producen cambios suaves en el espectrograma reconstruido.
- La interpolación "yes" $\rightarrow$ "no" muestra una **transición distinguible y coherente** entre ambas clases, lo que indica que el modelo capturó diferencias globales en la estructura espectral de cada palabra.
- A pesar de ello, el modelo no exhibe una separación perfecta entre clases (coherente con PCA y clustering), por lo que la continuidad observada es más “acústica” que “lingüística”.
- La perturbación con ruido revela la estabilidad local del manifold: cerca de z_{yes} las reconstrucciones son plausibles; lejos de él, pierden estructura.

7 Conclusiones y trabajo futuro

En este informe se implementó y analizó en detalle un autoencoder de audios aplicado al dataset *Speech Commands*. A partir de los experimentos realizados se pueden extraer las siguientes conclusiones principales:

- Es posible **comprimir** cada mel-espectrograma de 6464 dimensiones a un vector latente de sólo 32 dimensiones (factor de compresión $\approx 202\times$) manteniendo una calidad de reconstrucción razonable y una buena separación de clases en el espacio latente.
- El análisis exploratorio mediante PCA y clustering muestra que el autoencoder no sólo aprende a reconstruir, sino que también organiza los datos de forma coherente con sus etiquetas, revelando estructura latente útil.
- El modelo base muestra cierto comportamiento de **denoising** como efecto colateral, aunque un modelo específicamente diseñado para esta tarea debería ser entrenado con ejemplos ruidosos y limpios y, posiblemente, con una arquitectura más adecuada (por ejemplo, convolucional con *skip connections*).
- Las operaciones de interpolación y perturbación en el espacio latente sugieren que éste es relativamente continuo y semántico: desplazamientos suaves en z se traducen en cambios suaves en los audios generados, lo que abre la puerta a aplicaciones de **generación y edición de contenido de voz**.

Como líneas de trabajo futuro se podrían explorar:

- Autoencoders convolucionales o variacionales (VAE) para mejorar la calidad de las reconstrucciones y obtener un espacio latente con propiedades probabilísticas más controladas.

- Entrenamiento multi-tarea que combine reconstrucción, denoising y clasificación supervisada, de forma de explotar simultáneamente señales etiquetadas y no etiquetadas.
- Incorporar métricas perceptuales de calidad de audio y evaluar el impacto de la compresión latente desde la perspectiva del oyente humano.